HDOC DAY 53

PROBLEM

Write a C program that reads the size and elements of a variable-length integer array, nums[]. Pass the array to a function named product_except_self() using its address. Use pointers to pass the array between the calling function and product_except_self(). The function product_except_self() must:

- Create an output array, answer[], of the same size as nums[].
- Store in answer[i] the product of all elements of nums[] except nums[i].
- Ensure that the product does not include nums[i].
- Display the elements of answer[] in their original order, separated by commas.
- Avoid using division to calculate the products.
- Use the fact that the product of every prefix and suffix fits within a 32-bit integer.

CODE

```
#include <stdio.h>

#include <stdlib.h>


void product_except_self(int *nums, int n, int *answer)

{

  int prefix = 1;


  // Calculate products of all elements to the left

  for (int i = 0; i < n; i++)

  {

    answer[i] = prefix;

    prefix *= nums[i];

  }
```

```
    int suffix = 1;

    // Multiply by products of all elements to the right
    for (int i = n - 1; i >= 0; i--)
    {
        answer[i] *= suffix;
        suffix *= nums[i];
    }
}

int main()
{
    int n;

    scanf("%d", &n);

    int *nums = malloc(n * sizeof(int));
    int *answer = malloc(n * sizeof(int));

    if (nums == NULL || answer == NULL)
    {
        free(nums);
        free(answer);
        return 1;
    }

    for (int i = 0; i < n; i++)
    {
```

```
        scanf("%d", nums + i);
    }

    product_except_self(nums, n, answer);

    for (int i = 0; i < n; i++)
    {
        if (i > 0)
            printf(",");

        printf("%d", answer[i]);
    }

    printf("\n");

    free(nums);
    free(answer);

    return 0;
}
```